



Fracture

An intelligent system for detecting bone fractures from X-ray images.

Computer Vision Project

By

Leen Aburezeq and Dana Natsheh

Supervisor

Dr. Maria Yousef

June 5, 2026

Abstract

Abstract

This project focuses on detecting bone fractures from X-ray images using computer vision. Fractures can sometimes be small or unclear, which makes them hard to detect manually.

A dataset from Roboflow was used in this project. The dataset contains X-ray images with labeled fracture areas. Three deep learning models were trained and compared: YOLOv8, SegFormer-B2, and U-Net++ EfficientNet-B4. The goal was to detect the fracture area and show it using a segmentation mask.

The models were evaluated using metrics such as Dice Score, IoU, precision, recall, and mAP. The results showed that U-Net++ EfficientNet-B4 gave the best segmentation performance, while SegFormer-B2 achieved very close results. YOLOv8 had lower segmentation results compared with the other two models.

In conclusion, deep learning can help detect and segment fracture areas in X-ray images. This system can be used as a support tool to help doctors, but it does not replace medical diagnosis.

1 Introduction

Bone fractures are a common medical problem, and doctors usually use X-ray images to find them. But sometimes, it is not easy to see the fracture clearly, especially when the image is not very clear or the fracture is small.

In this project, we use computer vision to help detect bone fractures from X-ray images. We compare three different deep learning models to see which one performs better in detecting and segmenting bone fractures from X-ray images.

Instead of only saying if there is a fracture or not, our model can show the exact location of the fracture on the image. This makes it easier to understand and helps in faster and more accurate diagnosis. Fracture segmentation is considered a difficult task because fractures can be very small, unclear, or hidden in the X-ray image.

This project uses a dataset from Roboflow [1], which contains many X-ray images with labeled fracture areas. We train a model to learn from these images and then use it to detect fractures in new images.

1.1 Problem Statement

Detecting bone fractures from X-ray images is not always easy. Doctors need to carefully check each image, and sometimes the fracture can be small or not very clear. Small fractures are especially difficult for the model to detect, which makes fracture segmentation more challenging. This can take time, and in some cases, the fracture might be missed or noticed late. Another problem is that there are many X-ray images, especially in busy hospitals, so checking all of them manually can be tiring and slow. Also, different doctors may see the same image in different ways, which can lead to different results. In this project, we try to solve this problem by using computer vision. The goal is to build a system that can automatically look at an X-ray image and detect if there is a fracture, and more importantly, show where the fracture is located. We use a dataset from Roboflow that contains X-ray images with marked fracture areas. By training a model on this data, we aim to help make fracture detection faster, easier, and more consistent. This system is not meant to replace doctors, but to support them by giving a quick and clear idea about possible fracture areas in the image. The dataset used in this project is general and contains different types of fractures, not one specific fracture type. This increases the difficulty of the detection task because the fractures appear in different shapes and locations. In this research, we compare three deep learning models: YOLOv8, SegFormer-B2, and U-Net++ EfficientNet-B4, to study which model can handle these challenges better.

1.2 Objectives

- Build a system that can detect bone fractures from X-ray images automatically.
- Train a model using a labeled dataset from Roboflow.
- Make the model not only detect fractures, but also show the exact location of the fracture on the image.
- Improve the speed of checking X-ray images compared to manual checking.
- Create a simple and clear output that is easy to understand (showing the fracture area on the image).
- Compare three different deep learning models and evaluate which one gives better segmentation results.

1.3 Research Questions

- Can a computer vision model detect bone fractures from X-ray images correctly?
- How well can the model find the exact location of the fracture in the image?
- Can this system help make fracture detection faster and easier?
- Can this system be useful as a support tool for doctors?
- Which model gives better segmentation results for bone fracture detection?

1.4 Significance of the Study

This study is important because it shows how computer vision can help in medical image analysis, especially in detecting bone fractures from X-ray images. Instead of checking images manually, the system can give a quick result and highlight the fracture area.

The study can help save time and reduce effort, especially when there are many X-ray images to check. It can also help make the results more consistent and reduce human error.

Overall, this system can be used as a supportive tool to help doctors and improve the process of detecting fractures. The study also helps show the challenges of fracture segmentation, especially when using a general dataset with many different fracture types and small fracture areas.

2 Related Work

2.1 Introduction

This chapter presents the related work for fracture detection using X-ray images. Many researchers have worked on this topic because detecting bone fractures is important in medical diagnosis. X-ray images are commonly used by doctors, but reading them can sometimes take time and may include human error. For this reason, many studies used computer vision and deep learning methods to help detect fractures automatically. This chapter reviews previous studies that used CNN models, object detection models, segmentation methods, and medical X-ray datasets.

2.2 Traditional Image Processing Methods

Many previous studies focused on detecting bone fractures from X-ray images using computer vision

techniques. Early studies used traditional image processing methods such as grayscale conversion, noise removal, edge detection, segmentation, and feature extraction. After extracting features from the X-ray image, machine learning algorithms such as KNN and SVM were used to classify the image as fractured or normal. These methods showed that computer-aided systems can help doctors, but they usually depend on hand-made features, which may not work well for all types of X-ray images [2].

2.3 CNN-Based Fracture Detection

Later studies used Convolutional Neural Networks (CNNs) for fracture detection. CNN models are useful because they can automatically learn important features from images without the need for manual feature extraction. Several papers showed that CNN-based systems can achieve high accuracy in classifying X-ray images as fractured or non-fractured. These studies proved that deep learning can improve the speed and accuracy of fracture detection [3–5].

2.4 Medical X-ray Datasets

Many studies used large medical X-ray datasets to train and test their models. One of the most popular datasets is MURA, which contains many musculoskeletal X-ray images labeled as normal or abnormal. Another dataset is GRAZPEDWRI-DX, which focuses on wrist fractures and includes labeled images with bounding boxes and annotations. These datasets are important because deep learning models need a large amount of data to learn correctly and achieve good performance, such as MURA [6] and GRAZPEDWRI-DX [7].

2.5 Object Detection and YOLO Models

Other studies used object detection models to locate fractures inside X-ray images. Models such as Faster R-CNN, RetinaNet, YOLOv7, and YOLOv8 were tested on wrist X-ray images. These models do not only classify the image, but they can also show the fracture location using a bounding box. This is useful because it helps the doctor see where the fracture may exist. However, segmentation models provide more detailed results because they highlight the exact fracture area instead of only drawing a bounding box. Some studies showed that YOLO-based models are fast and suitable for real-time fracture detection [8, 9].

2.6 Segmentation Methods

Some studies used image segmentation methods to detect fractures. Segmentation divides the image into smaller regions and helps highlight the exact fracture area. This method provides more detailed results compared to classification and detection. However, it is more complex and requires accurate labeled data [10]. It is also difficult when fractures are very small or when the dataset contains many different fracture types.

2.7 Summary

In summary, previous studies showed that computer vision and deep learning can help in fracture detection from X-ray images. Traditional methods used image processing and machine learning, while recent methods used CNN, object detection, and segmentation models. Large datasets such as MURA and GRAZPEDWRI-DX helped improve model training and evaluation. These studies support the idea of our project, which aims to compare three deep learning models for fracture segmentation and build an effective fracture detection system using X-ray images.

3 Methodology

Dataset Source and Download Format The same dataset was used for all three models in this project. The dataset was downloaded from Roboflow in COCO segmentation format. It contains X-ray images with labeled fracture areas. After downloading the dataset, it was prepared differently depending on the model requirements. For YOLOv8, the COCO annotations were converted into YOLO segmentation format. For SegFormer-B2 and U-Net++ EfficientNet-B4, the COCO annotations were used to create binary fracture masks.

3.1 YOLOv8 Model

3.1.1 Data Cleaning

Before training the model, the dataset labels were checked and cleaned. All fracture annotations were converted into one class only, which is *fracture*. This helped make the dataset more consistent during training.

The annotation files were also checked to make sure all images had correct labels and annotation files.

3.1.2 Data Preparation

For YOLOv8, the annotation files were converted from COCO segmentation format into YOLO segmentation labels.

After conversion, the dataset folders were organized into:

- images/train
- images/valid
- images/test
- labels/train
- labels/valid
- labels/test

A *data.yaml* file was then created to define the dataset paths and class names.

3.1.3 Data Analysis

The dataset was analyzed to check the number of images and annotations in each set. The dataset contains 6,744 training images, 1,448 validation images, and 1,440 testing images. Random images and segmentation masks were displayed to visually inspect the fracture labels and make sure the dataset was prepared correctly.

3.1.4 Model Training

The YOLOv8 model was set to train for 150 epochs using image size 1024 and batch size 4. Early stopping was used, and the training stopped at epoch 121 when the model stopped improving.

During training, several augmentation techniques were applied such as:

- Rotation
- Scaling
- Translation
- Horizontal flipping
- Mosaic augmentation

3.1.5 YOLO Model

YOLO, which stands for You Only Look Once, is a deep learning model used for object detection and image segmentation. Unlike older models that scan an image multiple times in different regions, YOLO looks at the whole image just once and predicts where objects are at the same time. This makes it much

faster than traditional approaches. The version used in this project is YOLOv8, which is the eighth and most recent generation of YOLO. It improves on older versions by being more accurate, supporting segmentation tasks, and being easier to use. Specifically, the medium-sized version of YOLOv8 (yolov8m-seg) was used, which balances between speed and accuracy. Instead of training from scratch, the model was loaded with pretrained weights, meaning it had already l...

3.1.6 Model Evaluation

After training, the model performance was evaluated using validation images. Several evaluation metrics were used, including:

- Precision
- Recall
- F1-score
- mAP for bounding boxes
- mAP for segmentation masks

3.1.7 Prediction and Visualization

Finally, the trained model was tested on random X-ray images. The predicted fracture regions were displayed and compared with the original labeled masks. This helped visually evaluate how accurately the model segmented the fracture areas.

3.2 SegFormer-B2 Model

3.2.1 Dataset Collection

The dataset was divided into training, validation, and testing sets. It included 6,744 training images, 1,448 validation images, and 1,440 testing images.

3.2.2 Data Preprocessing

Before training the model, the images were prepared to improve the quality of the input. CLAHE was applied to the X-ray images to improve contrast and make fracture areas clearer.

The images were resized to 512×512 . The masks were also prepared from the COCO annotations, where each fracture annotation was converted into a binary mask. In this mask, the fracture area is marked and the background is ignored.

3.2.3 Data Augmentation

Data augmentation was used during training to help the model learn better from different image conditions. Several augmentation techniques were applied, such as:

- Horizontal flipping
- Rotation
- Shifting and scaling
- Brightness and contrast changes
- Gaussian noise

These techniques help the model become more flexible and reduce overfitting.

3.2.4 SegFormer-B2 Model

SegFormer-B2 is a pre-trained deep learning model used for semantic segmentation. In this project, the model was loaded using the pre-trained nvidia/mit-b2 weights. Using a pre-trained model helped the training process because the model already learned useful image features from a large dataset before fine-tuning it on the fracture X-ray dataset.

Unlike normal CNN models, SegFormer uses a Transformer-based encoder. This helps the model learn both small details and larger image patterns.

Training this model required a strong GPU and took many hours because segmentation models are computationally expensive. The model needed high memory and processing power to train on X-ray images and masks.

3.2.5 Model Training

The SegFormer-B2 model was trained using image size 512×512 and batch size 4. The model was trained for 80 epochs.

A combined loss function was used during training. This loss combined Tversky Loss and Focal Loss. Tversky Loss helps with small fracture areas, while Focal Loss helps the model focus more on difficult examples.

The AdamW optimizer was used for training. A learning rate scheduler was also used to control the learning rate during training.

3.2.6 Model Evaluation

After training, the model was evaluated using the test set. The evaluation metrics used were:

- Loss

- Dice Score
- IoU
- Precision
- Recall

These metrics were used to measure how close the predicted fracture mask was to the real fracture mask.

3.2.7 Prediction and Visualization

After evaluation, random test images were used to visually check the model predictions. The original X-ray image, the ground truth mask, and the predicted mask were displayed together.

This helped show how well the model was able to find the fracture area in the X-ray image.

3.3 U-Net++ EfficientNet-B4 Model

U-Net++ EfficientNet-B4 is a deep learning model used for image segmentation. It is based on the U-Net++ architecture, which is an improved version of the original U-Net model. U-Net++ helps the model learn more detailed features by using connected layers between the encoder and decoder parts.

In this project, EfficientNet-B4 was used as the encoder. The encoder is responsible for extracting important features from the X-ray images, such as edges, shapes, and fracture patterns. After that, the decoder uses these features to create a segmentation mask that shows the fracture area.

This model was used because it is suitable for medical image segmentation. It can help detect small and unclear fracture regions in X-ray images and show their location using a mask.

3.3.1 Training Results

The U-Net++ EfficientNet-B4 model was trained on the fracture X-ray dataset. During training, the model learned how to segment fracture areas from X-ray images.

The model was trained for 50 epochs using image size 512 and batch size 4. Although the best validation Dice score was achieved at epoch 14, the training was continued until 50 epochs to observe the full training behavior and check if the model could improve again in later epochs.

After around the twenties epochs, signs of overfitting started to appear. The training performance continued to improve, but the validation improvement became limited. This means the model was

learning the training data better, but it was not improving much on unseen validation data.

All trained models were saved to Google Drive during training to keep the best model weights and avoid losing the results. Early stopping was also used for all models to automatically stop training when the model stopped improving.

4 Results and Discussion

In general, all models faced similar challenges during fracture segmentation. The segmentation performance was lower in difficult cases, especially when:

- The fracture was very small.
- The X-ray image had low contrast.
- The fracture area was unclear.
- The fracture shape was complex.

4.1 YOLOv8

4.1.1 Training Results

The YOLOv8 segmentation model was successfully trained on the fracture X-ray dataset. During training, the model learned how to detect and segment fracture areas from different X-ray images.

The training process showed gradual improvement in segmentation performance over the epochs. The loss values decreased during training, which indicates that the model was learning the fracture patterns correctly.

4.1.2 Evaluation Results

After training, the model was evaluated using validation images. Several evaluation metrics were used, including precision, recall, F1-score, and segmentation mAP.

Table 1 shows the evaluation results of the YOLOv8 segmentation model.

Table 1: Evaluation results of YOLOv8 segmentation

model		
Metric	Bounding Box	Segmentation Mask
Precision	0.5650	0.5320
Recall	0.4115	0.3784
F1-Score	0.4762	0.4422
mAP50	0.4216	0.3845
mAP50-95	0.1798	0.1282

As shown in Table 1, the YOLOv8 model achieved better performance for bounding box detection than segmentation masks. The mask mAP50 was 0.3845, while the mask mAP50-95 was 0.1282. This means the model was able to detect some fracture regions, but the exact mask segmentation was still challenging, especially because fracture areas are small and unclear in many X-ray images.

The model achieved acceptable segmentation results on many X-ray images, especially when the fracture area was clear and visible. The model was able to highlight the fracture regions and generate segmentation masks around the fracture area.

These general challenges affected the YOLOv8 model performance, especially in difficult X-ray images.

4.1.3 Visualization Results

Several random X-ray images were tested after training. The predicted segmentation masks were visually checked with the ground truth masks from the dataset.

Figure 1 and Figure 2 show two examples of comparison results. Each example presents the original X-ray image, the model prediction, and the ground truth label. The results show that the model was able to detect the fracture region in many cases, although some predictions were smaller or less accurate than the original labeled area.



Figure 1: First example of YOLOv8 fracture segmentation comparison.

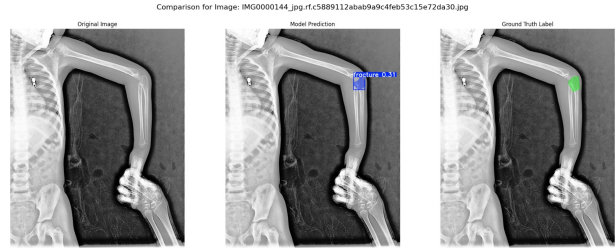


Figure 2: Second example of YOLOv8 fracture segmentation comparison.

The visualization results showed that the model could identify fracture regions in many images. In some cases, the predicted masks were close to the real masks. However, some predictions were incomplete or slightly shifted from the actual fracture location.

4.1.4 Discussion

Fracture segmentation is considered a difficult computer vision task because fractures can appear in many different shapes and sizes. Small fractures are especially challenging because they may not be clearly visible in the X-ray image.

Another challenge in this project is that the dataset is general and contains different types of fractures instead of one specific fracture type. This makes the segmentation task more difficult because the model must learn many fracture patterns.

Although the model did not achieve perfect segmentation results in all images, the system showed that deep learning and computer vision can help detect fracture regions automatically from X-ray images.

The results also showed that segmentation models can provide more detailed information compared to normal classification models because they highlight the exact fracture area instead of only classifying the image as fractured or normal.

4.2 SegFormer-B2

4.2.1 Training Results

The SegFormer-B2 model was successfully trained on the fracture X-ray dataset. During training, the model learned how to segment fracture regions from different X-ray images.

The training results showed stable performance in the last epochs. The best validation Dice score was 0.5786. This means the model was able to learn the fracture patterns, but the task was still difficult because fracture areas are usually small and not always clear.

4.2.2 Evaluation Results

After training, the model was tested on the test set. The model achieved the following results:

Table 2: Evaluation results of SegFormer-B2 model

Metric	Value
Test Loss	0.2493
Dice Score	0.5870
IoU	0.4323
Precision	0.5703
Recall	0.6416

The Dice score shows that the model had acceptable segmentation performance. The recall value was higher than the precision value, which means the model was able to find many fracture areas, but some predicted areas were not perfectly accurate.

4.2.3 Threshold Selection

Different threshold values were tested to choose the best value for converting the model output into a final mask. The best threshold was 0.55, with a Dice score of 0.5784 on the validation set.

This threshold was selected because it gave the best balance between detecting fracture areas and reducing incorrect predictions.

4.2.4 Visualization Results

The model predictions were displayed with the original X-ray images and the ground truth masks. The results showed that SegFormer-B2 was able to detect fracture regions in many test images.

In some cases, the predicted mask was close to the real mask. However, in difficult cases, the prediction was incomplete or not exactly aligned with the true fracture area.

These general challenges also affected the SegFormer-B2 model performance, especially in difficult X-ray images.

4.2.5 Training Curves

Figure 3 shows the training curves of the SegFormer-B2 model. The loss decreased during training, while the Dice score and IoU improved. This shows that the model was learning the fracture segmentation task.

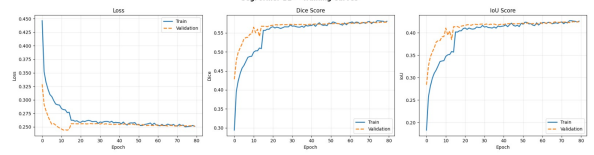


Figure 3: Training curves of the SegFormer-B2 model.

4.2.6 Prediction Examples

Figure 4 shows examples of the SegFormer-B2 predictions. The figure compares the original X-ray image, the ground truth mask, and the predicted mask. The model was able to detect fracture regions in several images, but some predictions were still small or incomplete.

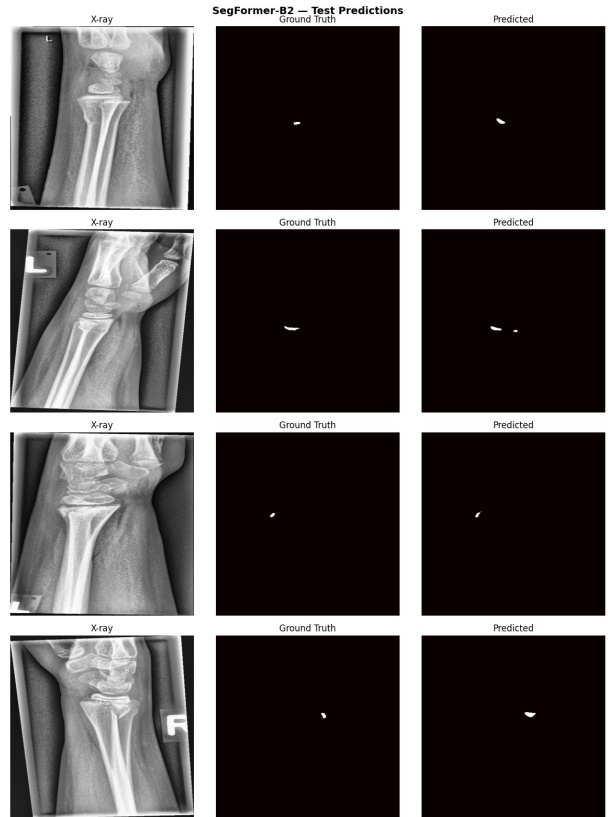


Figure 4: Prediction examples produced by the SegFormer-B2 model.

4.3 U-Net++

4.3.1 Training Results

The U-Net++ EfficientNet-B4 model was trained on the fracture X-ray dataset for 50 epochs using an image size of 512 and a batch size of 4. During training,

the model showed gradual improvement in segmentation performance. The training loss decreased from the early epochs until the final epochs, while the Dice score and IoU score increased.

The best validation Dice score achieved by the model was 0.5811. This result shows that the model was able to learn the fracture segmentation task, but the performance was still affected by the difficulty of the dataset. Since fracture regions are usually very small and sometimes unclear in X-ray images, achieving high segmentation accuracy is challenging.

The training curves also showed that the validation Dice and IoU became relatively stable in the last epochs. This indicates that the model reached a stable learning stage and was no longer improving significantly after the later epochs.

4.3.2 Evaluation Results

After training, the U-Net++ EfficientNet-B4 model was evaluated using the test set. The model achieved the following results:

Table 3: Evaluation results of U-Net++ EfficientNet-B4 model

Metric	Value
Test Loss	0.2396
Dice Score	0.5900
IoU	0.4369
Precision	0.5718
Recall	0.6464

The Dice score of 0.5900 shows that the model achieved acceptable segmentation performance. The IoU score of 0.4369 means that there was a moderate overlap between the predicted fracture masks and the ground truth masks. The recall value was 0.6464, which is higher than the precision value of 0.5718. This means that the model was able to detect many of the fracture areas, but some predicted regions were not perfectly accurate. In medical segmentation tasks, recall is important because missing a fracture area can be more critical than producing a slightly larger or imperfect prediction.

4.3.3 Visualization Results

The prediction results were displayed by comparing three images: the original X-ray image, the ground truth mask, and the predicted mask. The visualization examples showed that the model was able to detect fracture regions in several test images. In many cases, the predicted masks appeared close to the real fracture locations. However, the predicted regions

were sometimes smaller, slightly shifted, or incomplete compared to the ground truth masks. This is expected because the fracture areas in the dataset are very small and may not be clearly visible in the X-ray image.

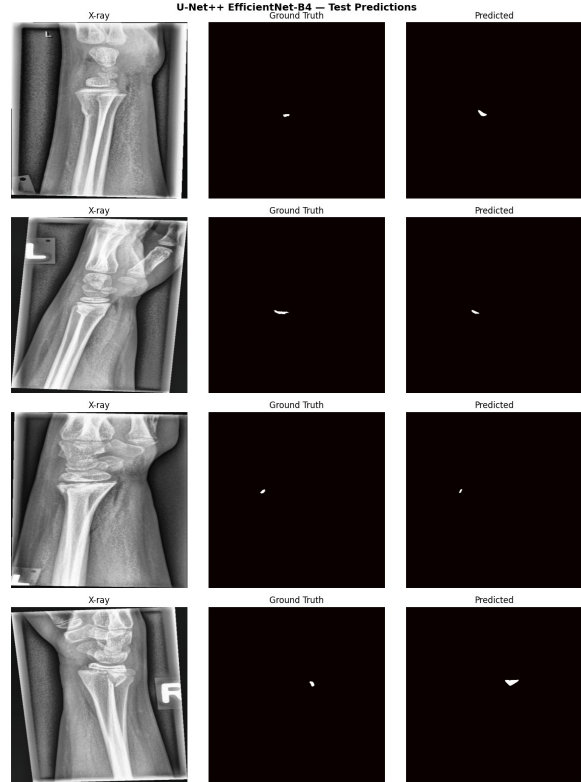


Figure 5: Prediction examples of U-Net++ EfficientNet-B4 on test X-ray images.

4.3.4 Training Curves

Figure 6 shows the training curves of the U-Net++ EfficientNet-B4 model. The loss curve decreased during training, while the Dice and IoU curves increased. This shows that the model was learning the fracture segmentation task over time.

The validation curves were more stable than the training curves in the final epochs, but a slight possible sign of overfitting can be noticed. The training performance continued to improve, while the validation improvement became limited near the end of training. However, this was not treated as severe overfitting because the validation results remained relatively stable.

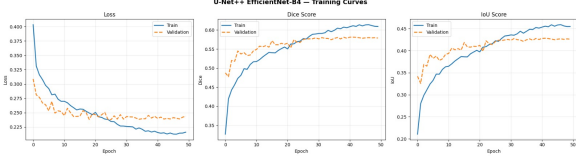


Figure 6: Training curves of U-Net++ EfficientNet-B4 model.

4.3.5 Discussion

The U-Net++ EfficientNet-B4 model achieved the best test Dice score among the tested segmentation models. The model reached a Dice score of 0.5900 and an IoU score of 0.4369 on the test set. These results show that the model performed well in detecting and segmenting fracture regions from X-ray images.

Compared with the SegFormer-B2 model, U-Net++ EfficientNet-B4 achieved a slightly higher Dice score. SegFormer-B2 achieved a Dice score of 0.5870, while U-Net++ EfficientNet-B4 achieved 0.5900. This means that U-Net++ EfficientNet-B4 performed slightly better in terms of overall mask similarity.

However, the difference between the models is small. This shows that the fracture segmentation task is difficult for all models because the fracture areas are often small, unclear, and different in shape and location. The general nature of the dataset also makes the task more challenging because the model must learn many different fracture patterns instead of focusing on one specific fracture type.

Overall, U-Net++ EfficientNet-B4 gave the best result in this project. The model was able to detect fracture regions and produce useful segmentation masks. However, the results are not perfect, and the model still needs improvement, especially for very small fractures, low-contrast X-ray images, and complex fracture shapes.

4.4 Model Comparison

Table 4 shows a comparison between the three segmentation models used in this project.

Table 4: Model comparison results

Model	Loss	Dice	IoU	Prec.	Rec.
YOLOv8	—	—	—	0.5320	0.3784
SegFormer-B2	0.2493	0.5870	0.4323	0.5703	0.6416
U-Net++ EfficientNet-B4	0.2396	0.5900	0.4369	0.5718	0.6464

From the comparison, U-Net++ EfficientNet-B4 achieved the best overall segmentation performance

in this project. It obtained the highest Dice score, IoU, precision, and recall compared with SegFormer-B2. This means that U-Net++ EfficientNet-B4 was slightly better at matching the real fracture masks.

SegFormer-B2 also achieved acceptable segmentation results, and its performance was very close to U-Net++ EfficientNet-B4. The difference between the two models was small, which shows that both models were able to learn the fracture segmentation task.

For YOLOv8, the evaluation format was different because YOLO reports both bounding box and segmentation mask metrics. In this comparison, the segmentation mask precision and recall were included. YOLOv8 was useful for detecting fracture regions, but its segmentation mask performance was lower than SegFormer-B2 and U-Net++ EfficientNet-B4. Therefore, based on our evaluation, U-Net++ EfficientNet-B4 was the best model for fracture segmentation, SegFormer-B2 was the second-best model, and YOLOv8 had the lowest segmentation performance.

Overall, the results show that fracture segmentation is a challenging task. This is because fracture regions are usually small, unclear, and difficult to identify clearly in X-ray images.

References

- [1] Roboflow, “Bone fracture detection dataset,” <https://universe.roboflow.com/capstone-project-qiyij/combined-dataset-seg-full-vkwbr/dataset/1>, 2024.
- [2] O. et al., “Detection of fracture bones in x-ray images categorization,” 2020.
- [3] M. et al., “Bone fracture detection using cnn,” 2024.
- [4] B. et al., “Bone fracture detection in x-ray images using cnn,” 2021.
- [5] H. et al., “Fracture detection using cnn and transfer learning,” 2025.
- [6] R. et al., “Mura: Large dataset for abnormality detection in musculoskeletal radiographs,” 2018.
- [7] J. et al., “A pediatric wrist trauma x-ray dataset (grazpedwri-dx),” *Scientific Data*, 2022.
- [8] Ju and Cai, “Fracture detection using yolov8,” 2023.
- [9] H. et al., “Fracture detection using deep learning-based object detection models,” *Sensors*, 2022.
- [10] T. et al., “Detecting distal radius fractures using segmentation-based model,” *Journal of Digital Imaging*, 2022.